

Unsupervised Learning with ANN

CSC3034 Computational Intelligence

Dr Tang Tiong Yew

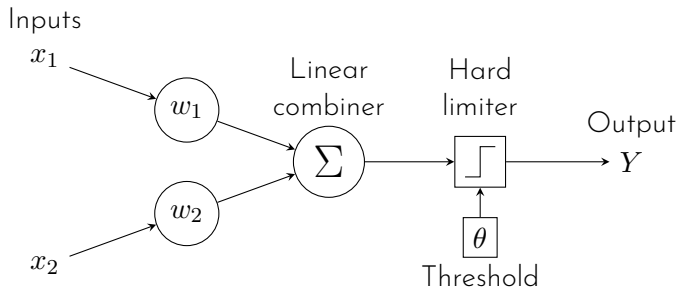
Recap

Unsupervised Learning

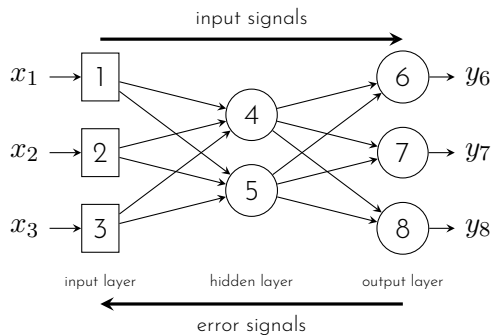
Hebbian learning

Competitive learning

Single-layer perceptron (Rosenblatt's perceptron)



Multilayer neural network with back-propagation learning algorithm



- ▶ The training data includes the input data and the desired output data.

- ▶ The training data includes the input data and the desired output data.
- ▶ The neural network is trained using the difference between the calculated output and the desired output as error.

- ▶ The training data includes the input data and the desired output data.
- ▶ The neural network is trained using the difference between the calculated output and the desired output as error.
- ▶ The aim of the neural network training is to minimise the discrepancy between the calculated output and the desired output.

Unsupervised learning

- ▶ Also known as self-organised learning.

Unsupervised learning

- ▶ Also known as self-organised learning.
- ▶ Only input data is provided for the purpose of training. There is no desired output data provided.

- ▶ Also known as self-organised learning.
- ▶ Only input data is provided for the purpose of training. There is no desired output data provided.
- ▶ The neural network is trained to discover significant features in the input data/signals/patterns, and learn how to classify the input data into different categories.

- ▶ Also known as self-organised learning.
- ▶ Only input data is provided for the purpose of training. There is no desired output data provided.
- ▶ The neural network is trained to discover significant features in the input data/signals/patterns, and learn how to classify the input data into different categories.
- ▶ Self-organising neural networks learn much faster than back-propagation networks, thus can be used in real time.

- ▶ Also known as self-organised learning.
- ▶ Only input data is provided for the purpose of training. There is no desired output data provided.
- ▶ The neural network is trained to discover significant features in the input data/signals/patterns, and learn how to classify the input data into different categories.
- ▶ Self-organising neural networks learn much faster than back-propagation networks, thus can be used in real time.
- ▶ Examples: Hebbian learning and competitive learning

- ▶ Proposed by a neuropsychologist Donald Hebb in 1949.

- ▶ Proposed by a neuropsychologist Donald Hebb in 1949.
- ▶ Hebb's law states that

- ▶ Proposed by a neuropsychologist Donald Hebb in 1949.
- ▶ Hebb's law states that
if **neuron** i is near enough to **excite neuron** j and repeatedly participates in its activation,

- ▶ Proposed by a neuropsychologist Donald Hebb in 1949.
- ▶ Hebb's law states that

if **neuron i** is near enough to **excite neuron j** and repeatedly participates in its activation,

the synaptic **connection** between these two neurons is **strengthened** and neuron j becomes more sensitive to stimuli from neuron i .

In ANN, Hebb's law is represented using two rules:

In ANN, Hebb's law is represented using two rules:

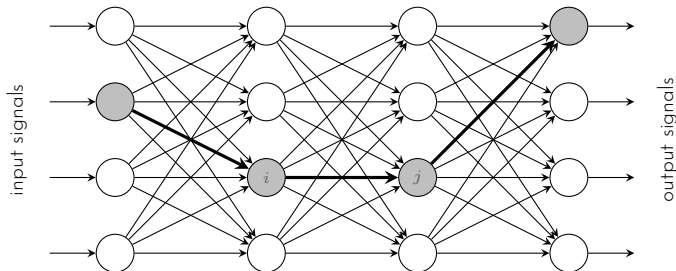
1. If two neurons on either side of a connection are activated **synchronously**, then the weight of that connection is **increased**.

In ANN, Hebb's law is represented using two rules:

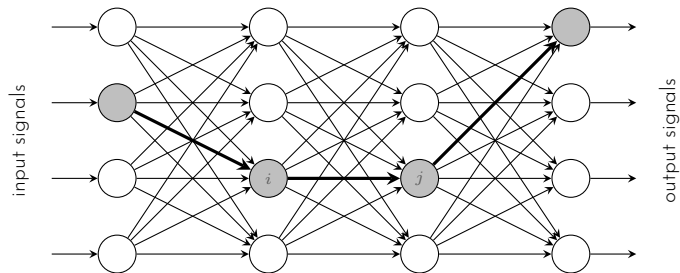
1. If two neurons on either side of a connection are activated **synchronously**, then the weight of that connection is **increased**.
2. If two neurons on either side of a connection are activated **asynchronously**, then the weight of that connection is **decreased**.

In ANN, Hebb's law is represented using two rules:

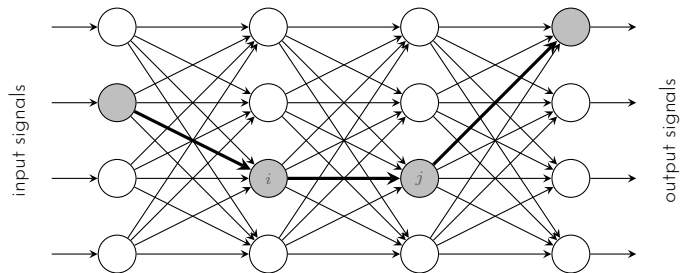
1. If two neurons on either side of a connection are activated **synchronously**, then the weight of that connection is **increased**.
2. If two neurons on either side of a connection are activated **asynchronously**, then the weight of that connection is **decreased**.



Representation of Hebb's law



Representation of Hebb's law



Activity product rule

$$\Delta w_{ij} = \alpha y_j x_i$$

where α is the learning rate

- ▶ The activity product rule ($\Delta w_{ij} = \alpha y_j x_i$) can only increase but not decrease the weights.

- ▶ The activity product rule ($\Delta w_{ij} = \alpha y_j x_i$) can only increase but not decrease the weights.
- ▶ Repeated application of input signals may drive the weights to saturation.

- ▶ The activity product rule ($\Delta w_{ij} = \alpha y_j x_i$) can only increase but not decrease the weights.
- ▶ Repeated application of input signals may drive the weights to saturation.
- ▶ To resolve this, a non-linear **forgetting factor** is introduced:

- ▶ The activity product rule ($\Delta w_{ij} = \alpha y_j x_i$) can only increase but not decrease the weights.
- ▶ Repeated application of input signals may drive the weights to saturation.
- ▶ To resolve this, a non-linear **forgetting factor** is introduced:

$$\Delta w_{ij} = \alpha y_j x_i - \phi y_j w_{ij}$$

- ▶ The activity product rule ($\Delta w_{ij} = \alpha y_j x_i$) can only increase but not decrease the weights.
- ▶ Repeated application of input signals may drive the weights to saturation.
- ▶ To resolve this, a non-linear **forgetting factor** is introduced:

$$\Delta w_{ij} = \alpha y_j x_i - \phi y_j w_{ij}$$

where ϕ is the forgetting factor, usually between 0 and 1, typically 0.01 to 0.1

Generalised activity product rule

The activity product rule with the forgetting rule can be generalised as the **generalised activity product rule**:

Generalised activity product rule

The activity product rule with the forgetting rule can be generalised as the **generalised activity product rule**:

$$\Delta w_{ij} = \phi y_j [\lambda x_i - w_{ij}]$$

Generalised activity product rule

The activity product rule with the forgetting rule can be generalised as the **generalised activity product rule**:

$$\Delta w_{ij} = \phi y_j [\lambda x_i - w_{ij}]$$

$$\text{where } \lambda = \frac{\alpha}{\phi}$$

Hebbian learning algorithm



Hebbian learning algorithm

Initialise weights w_{ij} and thresholds θ_j to small random values. Assign small positive values to α and ϕ .



Hebbian learning algorithm

Initialise weights w_{ij} and thresholds θ_j to small random values. Assign small positive values to α and ϕ .



Compute neuron output

$$y_j = \sum_{i=1}^n x_i w_{ij} - \theta_j$$

Hebbian learning algorithm

Initialise weights w_{ij} and thresholds θ_j to small random values. Assign small positive values to α and ϕ .



Compute neuron output

$$y_j = \sum_{i=1}^n x_i w_{ij} - \theta_j$$

Calculate weight correction using generalised activity product rule

$$\Delta w_{ij} = \phi y_j [\lambda x_i - w_{ij}]$$

Hebbian learning algorithm

Initialise weights w_{ij} and thresholds θ_j to small random values. Assign small positive values to α and ϕ .



Compute neuron output

$$y_j = \sum_{i=1}^n x_i w_{ij} - \theta_j$$

Calculate weight correction using generalised activity product rule

$$\Delta w_{ij} = \phi y_j [\lambda x_i - w_{ij}]$$

Update weights

$$w_{ij} = w_{ij} + \Delta w_{ij}$$

Hebbian learning algorithm

Initialise weights w_{ij} and thresholds θ_j to small random values. Assign small positive values to α and ϕ .

Repeat the process of

Activation - Weight training - Iteration

until the weights reach their steady-state values



Compute neuron output

$$y_j = \sum_{i=1}^n x_i w_{ij} - \theta_j$$

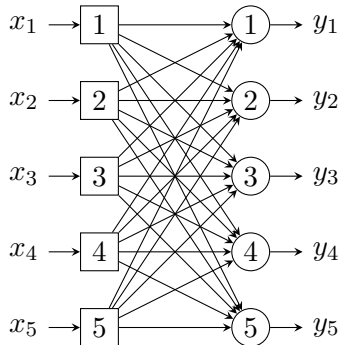
Calculate weight correction using generalised activity product rule

$$\Delta w_{ij} = \phi y_j [\lambda x_i - w_{ij}]$$

Update weights

$$w_{ij} = w_{ij} + \Delta w_{ij}$$

Example 1



Training data

$$X_1 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$X_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

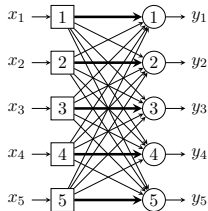
$$X_3 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

$$X_4 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

$$X_5 = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

Example 1

Initialisation



$$w_{ij} = \begin{array}{c|ccccc} & j & 1 & 2 & 3 & 4 & 5 \\ \hline i & 1 & 1 & 0 & 0 & 0 & 0 \\ & 2 & 0 & 1 & 0 & 0 & 0 \\ & 3 & 0 & 0 & 1 & 0 & 0 \\ & 4 & 0 & 0 & 0 & 1 & 0 \\ & 5 & 0 & 0 & 0 & 0 & 1 \end{array}$$

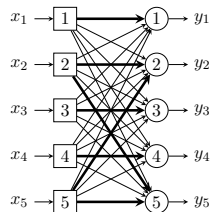
$$\theta_j = \begin{bmatrix} 0.4940 \\ 0.2661 \\ 0.0907 \\ 0.9478 \\ 0.0737 \end{bmatrix}$$

$$\alpha = 0.1$$

$$\phi = 0.02$$

Example 1

End of training (After 1270 iterations (254 epochs))



$$w_{ij} = \begin{matrix} & \begin{matrix} i \backslash j \end{matrix} & \begin{matrix} 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 4.9999 & 0 & 0 & 4.9998 \\ 0 & 0 & 4.9764 & 0 & 0 \\ 0 & 0 & 0 & 4.9764 & 0 \\ 0 & 4.9998 & 0 & 0 & 4.9999 \end{bmatrix} \end{matrix}$$

$$\theta_j = \begin{bmatrix} 0.4940 \\ 0.2661 \\ 0.0907 \\ 0.9478 \\ 0.0737 \end{bmatrix}$$

Example 1

Testing

$$X = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

$$Y = \text{step}(W^T X - \theta)$$

$$= \text{step} \left\{ \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 4.9999 & 0 & 0 & 4.9998 \\ 0 & 0 & 4.9764 & 0 & 0 \\ 0 & 0 & 0 & 4.9764 & 0 \\ 0 & 4.9998 & 0 & 0 & 4.9999 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} - \begin{bmatrix} 0.4940 \\ 0.2661 \\ 0.0907 \\ 0.9478 \\ 0.0737 \end{bmatrix} \right\}$$

$$= \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

- ▶ In Hebbian learning, several output neurons can be activated simultaneously.

Competitive learning

- ▶ In Hebbian learning, several output neurons can be activated simultaneously.
- ▶ In competitive learning, only a single output neuron is activated at any time.

- ▶ In Hebbian learning, several output neurons can be activated simultaneously.
- ▶ In competitive learning, only a single output neuron is activated at any time.
- ▶ The output neurons that wins the 'competition' is called the **winner-takes-all** neuron.

- ▶ In Hebbian learning, several output neurons can be activated simultaneously.
- ▶ In competitive learning, only a single output neuron is activated at any time.
- ▶ The output neurons that wins the 'competition' is called the **winner-takes-all** neuron.
- ▶ Example: **self-organising feature maps** proposed by Kohonen in 1989.

Self-organising feature maps

- ▶ In our cerebral cortex, different areas are responsible for different human activities, therefore, different areas are associated with different sensory inputs.

Self-organising feature maps

- ▶ In our cerebral cortex, different areas are responsible for different human activities, therefore, different areas are associated with different sensory inputs.
- ▶ Each sensory input is mapped onto a corresponding area in the cortex.

Self-organising feature maps

- ▶ In our cerebral cortex, different areas are responsible for different human activities, therefore, different areas are associated with different sensory inputs.
- ▶ Each sensory input is mapped onto a corresponding area in the cortex.
- ▶ Cortex is a self-organising computational map in our brain.

SUNWAY
UNIVERSITY

Jeffrey Cheah
Foundation

Nurturing the Seeds of Wisdom

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

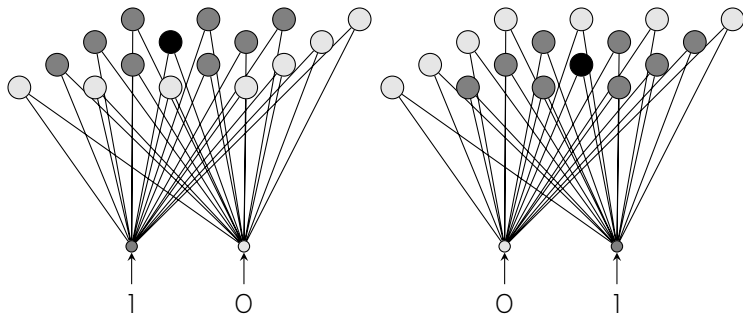
- ▶ Kohonen formulated the **principle of topographic map formation** (Kohonen, 1990).
 - the spatial location of an output neuron in the topographic map corresponds to a particular feature of the input pattern.

SUNWAY UNIVERSITY Jeffrey Cheah Foundation
Nurturing the Seeds of Wisdom

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ▶ ≡ ▶

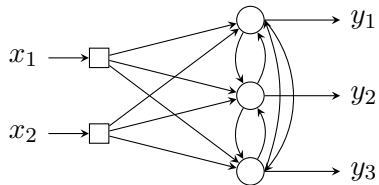
Self-organising feature map

- ▶ Kohonen formulated the **principle of topographic map formation** (Kohonen, 1990).
 - the spatial location of an output neuron in the topographic map corresponds to a particular feature of the input pattern.
- ▶ Kohonen also proposed the **feature-mapping model**.

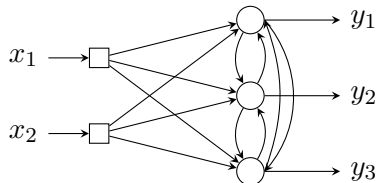


Kohonen network

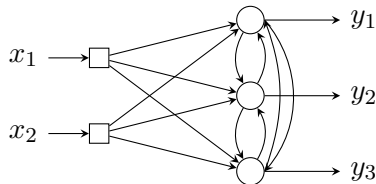
- Kohonen network consists of



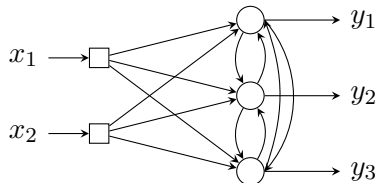
- ▶ Kohonen network consists of
 - a single layer of computation neurons,



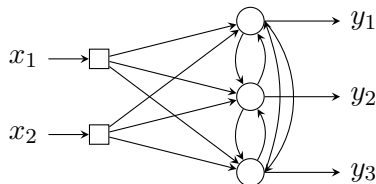
- ▶ Kohonen network consists of
 - a single layer of computation neurons,
 - forward connections (from input neurons to output neurons), and



- ▶ Kohonen network consists of
 - a single layer of computation neurons,
 - forward connections (from input neurons to output neurons), and
 - lateral connections (between output neurons).



- ▶ Kohonen network consists of
 - a single layer of computation neurons,
 - forward connections (from input neurons to output neurons), and
 - lateral connections (between output neurons).



- ▶ Lateral connections are used to create competition between neurons.

- ▶ The winning neuron is the neuron that its weight vector W best matches the input vector X .

- ▶ The winning neuron is the neuron that its weight vector W best matches the input vector X .
- ▶ Example: minimum-distance Euclidean criterion

$$j_X = \min_j ||X - W_j|| = \min_j \sqrt{\sum_{i=1}^n [x_i - w_{ij}]^2}$$

- ▶ Only the winning neuron and its neighbourhood are allowed to learn.

- ▶ Only the winning neuron and its neighbourhood are allowed to learn.
- ▶ The lateral connections are used to define the distance between the neurons and identify which neurons are in the winner's neighbourhood.

- ▶ Only the winning neuron and its neighbourhood are allowed to learn.
- ▶ The lateral connections are used to define the distance between the neurons and identify which neurons are in the winner's neighbourhood.
- ▶ There is no learning in the neurons that are not activated (i.e. not winner or winner's neighbour).

SUNWAY UNIVERSITY **Jeffrey Cheah Foundation**
Nurturing the Seeds of Wisdom

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

- ▶ Only the winning neuron and its neighbourhood are allowed to learn.
- ▶ The lateral connections are used to define the distance between the neurons and identify which neurons are in the winner's neighbourhood.
- ▶ There is no learning in the neurons that are not activated (i.e. not winner or winner's neighbour).
- ▶ The **standard competitive learning rule** is

$$\Delta w_{ij} = \begin{cases} \alpha(x_i - w_{ij}), & \text{if neuron } j \text{ is the winner or a winner's neighbour} \\ 0, & \text{otherwise} \end{cases}$$

- ▶ Only the winning neuron and its neighbourhood are allowed to learn.
- ▶ The lateral connections are used to define the distance between the neurons and identify which neurons are in the winner's neighbourhood.
- ▶ There is no learning in the neurons that are not activated (i.e. not winner or winner's neighbour).
- ▶ The **standard competitive learning rule** is

$$\Delta w_{ij} = \begin{cases} \alpha(x_i - w_{ij}), & \text{if neuron } j \text{ is the winner or a winner's neighbour} \\ 0, & \text{otherwise} \end{cases}$$

where α is the learning rate parameter, lies between 0 and 1.

- ▶ The overall effect of this competitive learning rule is to move the weight vector W_j of the winning neuron j towards the input pattern X .

Learning algorithm for Kohonen network



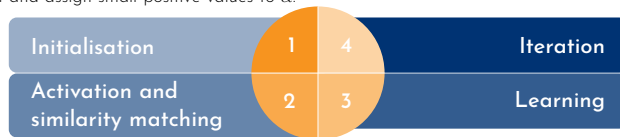
Learning algorithm for Kohonen network

Initialise weights w_{ij} to small random values between 0 and 1 and assign small positive values to α .



Learning algorithm for Kohonen network

Initialise weights w_{ij} to small random values between 0 and 1 and assign small positive values to α .



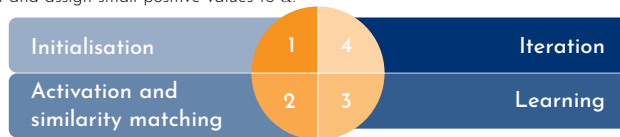
Apply input vector X and identify winner-takes-all neuron j_X

Example: minimum-distance Euclidean criterion

$$j_X = \min_j \|X - W_j\| = \min_j \sqrt{\sum_{i=1}^n [x_i - w_{ij}]^2}$$

Learning algorithm for Kohonen network

Initialise weights w_{ij} to small random values between 0 and 1 and assign small positive values to α .



Apply input vector X and identify winner-takes-all neuron j_X

Example: minimum-distance Euclidean criterion

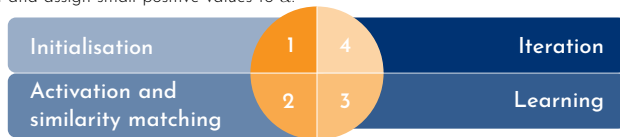
$$j_X = \min_j \|X - W_j\| = \min_j \sqrt{\sum_{i=1}^n [x_i - w_{ij}]^2}$$

Calculate weight correction

$$\Delta w_{ij} = \begin{cases} \alpha(x_i - w_{ij}), & j \in \Lambda_j \\ 0, & \text{otherwise} \end{cases}$$

Learning algorithm for Kohonen network

Initialise weights w_{ij} to small random values between 0 and 1 and assign small positive values to α .



Apply input vector X and identify winner-takes-all neuron j_X

Example: minimum-distance Euclidean criterion

$$j_X = \min_j \|X - W_j\| = \min_j \sqrt{\sum_{i=1}^n [x_i - w_{ij}]^2}$$

Calculate weight correction

$$\Delta w_{ij} = \begin{cases} \alpha(x_i - w_{ij}), & j \in \Lambda_j \\ 0, & \text{otherwise} \end{cases}$$

Update weights

$$w_{ij} = w_{ij} + \Delta w_{ij}$$

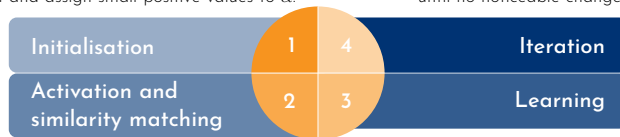
Learning algorithm for Kohonen network

Initialise weights w_{ij} to small random values between 0 and 1 and assign small positive values to α .

Repeat the process of

Activation - Learning

until no noticeable changes occur in the feature map



Apply input vector X and identify winner-takes-all neuron j_X

Example: minimum-distance Euclidean criterion

$$j_X = \min_j \|X - W_j\| = \min_j \sqrt{\sum_{i=1}^n [x_i - w_{ij}]^2}$$

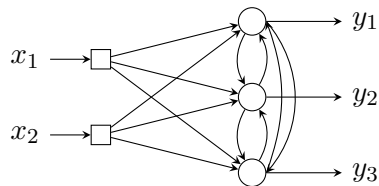
Calculate weight correction

$$\Delta w_{ij} = \begin{cases} \alpha(x_i - w_{ij}), & j \in \Lambda_j \\ 0, & \text{otherwise} \end{cases}$$

Update weights

$$w_{ij} = w_{ij} + \Delta w_{ij}$$

Example 2



Input vector

$$X = \begin{bmatrix} 0.52 \\ 0.12 \end{bmatrix}$$

Initial weight vectors

$$W_1 = \begin{bmatrix} 0.27 \\ 0.81 \end{bmatrix}$$

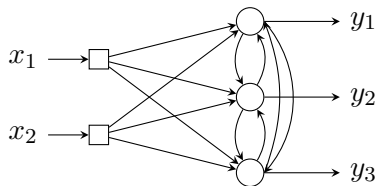
$$W_2 = \begin{bmatrix} 0.42 \\ 0.70 \end{bmatrix}$$

$$W_3 = \begin{bmatrix} 0.43 \\ 0.21 \end{bmatrix}$$

- Identify the winning neuron using the minimum-distance Euclidean criterion.

$$d_j = \sqrt{\sum_{i=1}^n [x_i - w_{ij}]^2}$$

Example 2



Input vector

$$X = \begin{bmatrix} 0.52 \\ 0.12 \end{bmatrix}$$

Initial weight vectors

$$W_1 = \begin{bmatrix} 0.27 \\ 0.81 \end{bmatrix}$$

$$W_2 = \begin{bmatrix} 0.42 \\ 0.70 \end{bmatrix}$$

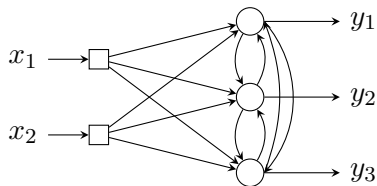
$$W_3 = \begin{bmatrix} 0.43 \\ 0.21 \end{bmatrix}$$

- Identify the winning neuron using the minimum-distance Euclidean criterion.

$$d_j = \sqrt{\sum_{i=1}^n [x_i - w_{ij}]^2}$$

$$d_1 = \sqrt{(x_1 - w_{11})^2 + (x_2 - w_{21})^2} = \sqrt{(0.52 - 0.27)^2 + (0.12 - 0.81)^2} = 0.73$$

Example 2



Input vector

$$X = \begin{bmatrix} 0.52 \\ 0.12 \end{bmatrix}$$

Initial weight vectors

$$W_1 = \begin{bmatrix} 0.27 \\ 0.81 \end{bmatrix}$$

$$W_2 = \begin{bmatrix} 0.42 \\ 0.70 \end{bmatrix}$$

$$W_3 = \begin{bmatrix} 0.43 \\ 0.21 \end{bmatrix}$$

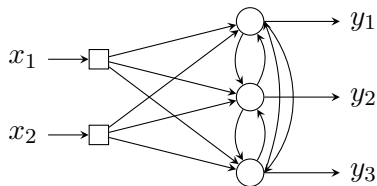
- Identify the winning neuron using the minimum-distance Euclidean criterion.

$$d_j = \sqrt{\sum_{i=1}^n [x_i - w_{ij}]^2}$$

$$d_1 = \sqrt{(x_1 - w_{11})^2 + (x_2 - w_{21})^2} = \sqrt{(0.52 - 0.27)^2 + (0.12 - 0.81)^2} = 0.73$$

$$d_2 = \sqrt{(x_1 - w_{12})^2 + (x_2 - w_{22})^2} = \sqrt{(0.52 - 0.42)^2 + (0.12 - 0.70)^2} = 0.59$$

Example 2



Input vector

$$X = \begin{bmatrix} 0.52 \\ 0.12 \end{bmatrix}$$

Initial weight vectors

$$W_1 = \begin{bmatrix} 0.27 \\ 0.81 \end{bmatrix}$$

$$W_2 = \begin{bmatrix} 0.42 \\ 0.70 \end{bmatrix}$$

$$W_3 = \begin{bmatrix} 0.43 \\ 0.21 \end{bmatrix}$$

- Identify the winning neuron using the minimum-distance Euclidean criterion.

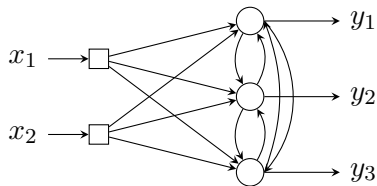
$$d_j = \sqrt{\sum_{i=1}^n [x_i - w_{ij}]^2}$$

$$d_1 = \sqrt{(x_1 - w_{11})^2 + (x_2 - w_{21})^2} = \sqrt{(0.52 - 0.27)^2 + (0.12 - 0.81)^2} = 0.73$$

$$d_2 = \sqrt{(x_1 - w_{12})^2 + (x_2 - w_{22})^2} = \sqrt{(0.52 - 0.42)^2 + (0.12 - 0.70)^2} = 0.59$$

$$d_3 = \sqrt{(x_1 - w_{13})^2 + (x_2 - w_{23})^2} = \sqrt{(0.52 - 0.43)^2 + (0.12 - 0.21)^2} = 0.13$$

Example 2



Input vector

$$X = \begin{bmatrix} 0.52 \\ 0.12 \end{bmatrix}$$

Initial weight vectors

$$W_1 = \begin{bmatrix} 0.27 \\ 0.81 \end{bmatrix}$$

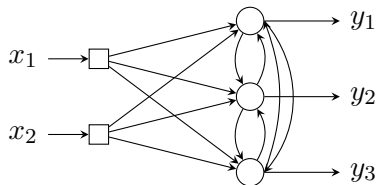
$$W_2 = \begin{bmatrix} 0.42 \\ 0.70 \end{bmatrix}$$

$$W_3 = \begin{bmatrix} 0.43 \\ 0.21 \end{bmatrix}$$

- Neuron 3 is the winner therefore its weight vector W_3 is updated.

$$\Delta w_{ij} = \alpha(x_i - w_{ij})$$

Example 2



Input vector

$$X = \begin{bmatrix} 0.52 \\ 0.12 \end{bmatrix}$$

Initial weight vectors

$$W_1 = \begin{bmatrix} 0.27 \\ 0.81 \end{bmatrix}$$

$$W_2 = \begin{bmatrix} 0.42 \\ 0.70 \end{bmatrix}$$

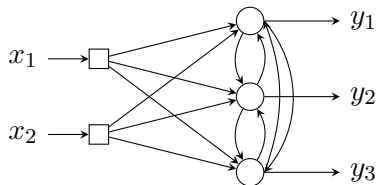
$$W_3 = \begin{bmatrix} 0.43 \\ 0.21 \end{bmatrix}$$

- Neuron 3 is the winner therefore its weight vector W_3 is updated.

$$\Delta w_{ij} = \alpha(x_i - w_{ij})$$

$$\Delta w_{13} = \alpha(x_1 - w_{13}) = 0.1(0.52 - 0.43) = 0.009$$

Example 2



Input vector

$$X = \begin{bmatrix} 0.52 \\ 0.12 \end{bmatrix}$$

Initial weight vectors

$$W_1 = \begin{bmatrix} 0.27 \\ 0.81 \end{bmatrix}$$

$$W_2 = \begin{bmatrix} 0.42 \\ 0.70 \end{bmatrix}$$

$$W_3 = \begin{bmatrix} 0.43 \\ 0.21 \end{bmatrix}$$

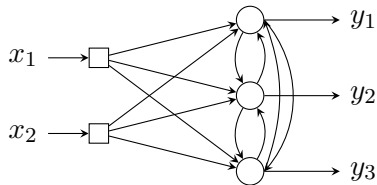
- Neuron 3 is the winner therefore its weight vector W_3 is updated.

$$\Delta w_{ij} = \alpha(x_i - w_{ij})$$

$$\Delta w_{13} = \alpha(x_1 - w_{13}) = 0.1(0.52 - 0.43) = 0.009$$

$$\Delta w_{23} = \alpha(x_2 - w_{23}) = 0.1(0.12 - 0.21) = -0.009$$

Example 2



Input vector

$$X = \begin{bmatrix} 0.52 \\ 0.12 \end{bmatrix}$$

Initial weight vectors

$$W_1 = \begin{bmatrix} 0.27 \\ 0.81 \end{bmatrix} \quad W_2 = \begin{bmatrix} 0.42 \\ 0.70 \end{bmatrix} \quad W_3 = \begin{bmatrix} 0.43 \\ 0.21 \end{bmatrix}$$

- Neuron 3 is the winner therefore its weight vector W_3 is updated.

$$\Delta w_{ij} = \alpha(x_i - w_{ij})$$

$$\Delta w_{13} = \alpha(x_1 - w_{13}) = 0.1(0.52 - 0.43) = 0.009$$

$$\Delta w_{23} = \alpha(x_2 - w_{23}) = 0.1(0.12 - 0.21) = -0.009$$

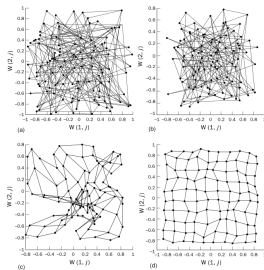
$$W_3 = W_3 + \Delta W_3 = \begin{bmatrix} 0.43 \\ 0.21 \end{bmatrix} + \begin{bmatrix} 0.009 \\ -0.009 \end{bmatrix} = \begin{bmatrix} 0.439 \\ 0.201 \end{bmatrix}$$

Example 3

Consider a Kohonen network with 100 neurons (10 rows and 10 columns).

Example 3

Consider a Kohonen network with 100 neurons (10 rows and 10 columns).



- (a) initial random weights
- (b) network after 100 iterations
- (c) network after 1000 iterations
- (d) network after 10,000 iterations

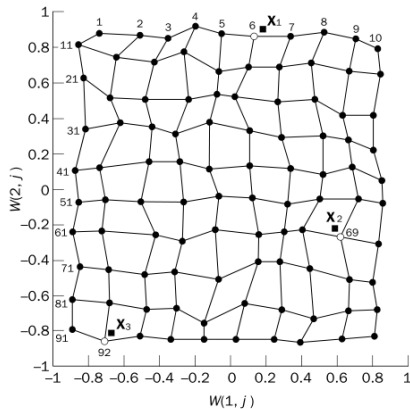
Example 3

If we apply the following input vectors,

$$X_1 = \begin{bmatrix} 0.2 \\ 0.9 \end{bmatrix}$$

$$X_2 = \begin{bmatrix} 0.6 \\ -0.2 \end{bmatrix}$$

$$X_3 = \begin{bmatrix} -0.7 \\ -0.8 \end{bmatrix}$$



Unsupervised learning

Unsupervised learning

- ▶ Hebbian learning

Unsupervised learning

- ▶ Hebbian learning - Hebbian network

Unsupervised learning

- ▶ Hebbian learning - Hebbian network
- ▶ Competitive learning

Unsupervised learning

- ▶ Hebbian learning - Hebbian network
- ▶ Competitive learning - Kohonen network



SUNWAY
UNIVERSITY

Thank you

CSC3034 Current Course Update

07 Unsupervised Learning - presentation

Synchronized from the current CSC3034 course site on 14 August 2026.

- Current Lab 6 trains and evaluates a scikit-learn MLP on the Wine dataset.
- Current Lab 7 visualizes neural-network hyperplanes on standardized Iris features.
- Current Lab 8 adds Keras CNN/LSTM examples and Isaac Sim vision demonstrations.

Current materials author: Dr Tang Tiong Yew

See the synchronized lab notes and runnable examples for full instructions.